

TBOT BRAIN REBUILD — COMPLETE CONSOLIDATED IMPLEMENTATION PLAN

Everything in one file, ordered so nothing later depends on something not yet done above it.

READ THIS FIRST, STEPHEN

- **Everything in this file goes live on day one — this is a deliberate call, not an oversight.** A bot that isn't trading has its own real cost, and that cost is why this whole rebuild started. The tradeoff being accepted here, in plain terms: bugs get found by watching the bot trade and lose or win real money, not by a pre-launch comparison window. That's a real, informed decision, not a mistake to be walked back quietly later.
- **The order of the six parts below is still not arbitrary, even though nothing is gated anymore.** Part 0 still has to go first — it's plumbing that Parts 1, 2, and 5 all silently depend on. Skipping it

means later parts throw `AttributeError` the first time they run, live, with real capital behind them — which is a much worse time to find that out than in review.

- **Part 5 changed jobs.** It used to be the gate deciding *whether* Parts 2–4 were allowed to go live. It's now the monitoring layer watching them *while* they're live from hour one — same checks, same code, pointed at a running system instead of a pending decision.
- **The single most important thing in this whole file, given the approach: 5.1's `tier3_shadow_enabled` flag is the emergency brake.** Flipping it back to `True` instantly reverts every Tier 3 judge to the old, already-proven behavior — no redeploy, no code change, one config value. If the daily digest or the live divergence comparison (5.2–5.4, still running, just no longer gating anything) shows a judge dead, pinned, or clearly losing against the old version, that flag is the fast, safe first move while the actual bug gets fixed properly — not a full rollback of everything in this file, just the one piece that's misbehaving.
- **`per_tick_livermore_enabled` and `conviction_dying_directional_enabled` both flip to `True` in Part 1/Part 2's config, per this same decision.** Both were left off by a previous session specifically pending a 21-day observation

window that was never run. Flipping them now means that observation happens live instead of beforehand — worth knowing precisely which two flags carry that specific, previously-named risk, not just "everything's live now" as an undifferentiated mass.

- **Part 6 (the zone ladder) still isn't part of this deployment.** Not a caution about risk — it just isn't wired into the live pipeline at all yet. Nothing to flip.
- Every fix below was checked against the actual repo before being written down. Two passes were run on the whole set: one checking every fix's own logic, one checking that anything claiming to connect to something else actually can. That verification didn't change — what changed is when bugs beyond that get caught: live, not in a shadow window first.

PART 0 — FOUNDATION (apply first, everything else depends on this)

0.1 — `OutcomeTracker`, the class three other fixes assume exists

Why: X.2 (Part 1), and RLHF's H1/H2 (Part 1) all call methods on `self.outcome_pipeline`. It was never

built. Every one of those calls would throw `AttributeError` today.

Where: new file or top of an existing shared module — wherever `HealthMonitor` already lives is a reasonable home.

```
class OutcomeTracker:
    """The real shared pipeline behind Tier
    6.2. One class, one name,
    serving X.2's softening events, near-
    miss tracking, and RLHF's human
    labels — instead of each one quietly
    inventing its own logger."""
    def __init__(self):
        self._pending = []
        self._records = []

    def tag(self, asset, direction, price,
            timestamp, event_type="softening"):
        self._pending.append({
            "asset": asset, "direction":
            direction, "price": price,
            "timestamp": timestamp,
            "event_type": event_type, "resolved":
            False,
        })

    def check_due(self, price_lookup_fn,
                  bars_ahead=6):
        for entry in self._pending:
            if entry["resolved"]:
                continue
```

```

        later_price =
price_lookup_fn(entry["asset"],
entry["timestamp"], bars_ahead)
        if later_price is None:
            continue
        entry["moved_as_expected"] = (
            (later_price >
entry["price"]) if entry["direction"] ==
"bullish" else (later_price <
entry["price"]))
        )
        entry["resolved"] = True

    def record(self, position, vtm, reason,
human_label=None):
        self._records.append({
            "trade_id": getattr(position,
"db_trade_id", None),
            "asset": getattr(vtm, "asset",
None),
            "exit_reason": reason,
            "human_label": human_label,
        })

    def attach_human_label(self, trade_id,
label):
        for rec in self._records:
            if rec["trade_id"] == trade_id:
                rec["human_label"] = label
                return True
        return False

```

Layman: one shared notebook. Every fix in this whole file that needs to remember "I did X, check later if X was right" writes to this same notebook instead of each one keeping its own.

0.2 — Inject `outcome_pipeline` into every class that actually calls it

Why, found by checking rather than assuming:

`outcome_pipeline` was referenced from three different classes across this session's fixes, and none of them actually receive it. `PortfolioManager` already has an established, working precedent for this exact kind of injection (`self.aggregators = aggregators` , tagged # 04c in the real code) — follow that same shape, don't invent a new one.

Where — instantiate once, in `main.py` , alongside

```
self.health_monitor = HealthMonitor():
```

```
self.outcome_pipeline = OutcomeTracker()
```

Then inject it at every real instantiation site —

confirmed these exact call sites exist in `main.py` :

```
# main.py:598 — PortfolioManager(...)
self.portfolio_manager = PortfolioManager(
    ..., aggregators=self.aggregators,
```

```
outcome_pipeline=self.outcome_pipeline,  
)
```

```
# main.py:835 – TradingTelegramBot(...)  
self.telegram_bot = TradingTelegramBot(  
    ...,  
    outcome_pipeline=self.outcome_pipeline,  
)
```

```
# main.py – ALL FOUR  
InstitutionalCouncilAggregator(...) call  
sites: lines 1375, 1466, 2918, 2964.  
# Every one needs this added – missing even  
one means some assets silently  
# don't get the fix while others do.  
InstitutionalCouncilAggregator(  
    ...,  
    outcome_pipeline=self.outcome_pipeline,  
)
```

And the receiving end of each constructor:

```
# InstitutionalCouncilAggregator.__init__ –  
add alongside the existing  
# mtf_integration=None,  
performance_tracker=None injections:  
def __init__(self, ...,  
    mtf_integration=None,  
    performance_tracker=None,  
        outcome_pipeline=None,  
    use_macro_governor=True,
```

```
use_gatekeeper=True):  
    ...  
    self.mtf_integration = mtf_integration  
    self.performance_tracker =  
performance_tracker  
    self.outcome_pipeline =  
outcome_pipeline
```

```
# PortfolioManager.__init__ - add  
outcome_pipeline alongside the aggregators  
param it already has:  
def __init__(self, ..., aggregators=None,  
outcome_pipeline=None):  
    self.aggregators = aggregators  
    self.outcome_pipeline =  
outcome_pipeline
```

```
# TradingTelegramBot.__init__ - add:  
def __init__(self, ...,  
outcome_pipeline=None):  
    self.outcome_pipeline =  
outcome_pipeline
```

Layman: the shared notebook from 0.1 doesn't do anything until three specific people (Council, the position manager, the Telegram bot) are actually handed a copy of it. This is that handoff, at every place they get created — four places for Council alone, since it gets built fresh per asset.

0.3 – Inject telegram into VTM (a three-level thread, not one line)

Why: 4.5's alert layer (Part 3) calls

```
self.telegram.is_running /  
self.telegram.send_notification(...)
```

VTM has no such attribute – and neither does

`PortfolioManager`, which is what actually creates VTM instances. The real reference only exists on the main bot class. This has to be threaded down two levels, not patched at VTM alone.

```
# main.py – wherever the real telegram bot  
handle already exists, pass it  
# into PortfolioManager alongside  
aggregators/outcome_pipeline from 0.2:  
self.portfolio_manager = PortfolioManager(  
    ..., aggregators=self.aggregators,  
    outcome_pipeline=self.outcome_pipeline,  
    telegram=self.telegram_bot,  
)
```

```
# PortfolioManager.__init__ – receive and  
hold it:  
def __init__(self, ..., aggregators=None,  
    outcome_pipeline=None, telegram=None):  
    self.aggregators = aggregators  
    self.outcome_pipeline =  
outcome_pipeline  
    self.telegram = telegram
```

```

# Wherever PortfolioManager actually
constructs a new VTM instance
# (position open), pass it through the
final leg:
new_vtm = VeteranTradeManager(
    ..., structure_levels_ref=...,
    entry_retest_type=..., # already there
    from batch-4's item 5
    telegram=self.telegram,
)

```

```

# VeteranTradeManager.__init__ – receive
it, alongside the existing structure
params:
def __init__(self, ...,
    structure_levels_ref=None,
    entry_retest_type=None, telegram=None):
    self.structure_levels_ref =
    structure_levels_ref
    self.entry_retest_type =
    entry_retest_type
    self.telegram = telegram

```

Layman: VTM needs to be able to send a Telegram message, but it never got handed a phone. This gives it one, passed hand-to-hand from the top of the bot down to the specific trade manager that needs it.

0.4 – Inject aggregators into both exchange handlers

Why: 4.4's continuous monitoring (Part 3) needs a fresh council call each cycle –

```
self.aggregators[asset_name]
["council"].get_aggregated_signal(...) – from
inside mt5_handler.py and binance_handler.py .
Confirmed: neither class has self.aggregators .
Only the main bot class does.
```

```
# main.py:709 – MT5ExecutionHandler(...)
self.mt5_handler = MT5ExecutionHandler(...,
aggregators=self.aggregators)
```

```
# Wherever BinanceHandler gets constructed,
same addition:
self.binance_handler = BinanceHandler(...,
aggregators=self.aggregators)
```

```
# Both classes' __init__:
def __init__(self, ..., aggregators=None):
    self.aggregators = aggregators
```

Layman: the two places that actually place trades need to be able to ask "what do the judges think, right now" – and they never had a way to ask. This gives them the same reference the rest of the bot already has.

0.5 – BOS/CHoCH swing-detection fix (Tier X.1) – the foundation Tier 2.1, the zone ladder, and Tier 6.1 all reuse

Why: four real problems in one function – no limit on how far back it searches for "recent," one direction's signal silently blocked the other's, it measured wicks instead of closes (against your own stated rule), and it failed completely silently.

Where: `src/execution/signal_aggregator.py`, the swing-high/swing-low block.

```
try:
    if len(df) < 20:
        return

    highs_arr = df["high"].values
    lows_arr = df["low"].values
    closes_arr = df["close"].values

    _atr_raw = float(df["atr"].iloc[-1])
    if "atr" in df.columns else 0.0
    _min_depth = 0.3 * _atr_raw if _atr_raw
    > 0 else 0.0

    MAX_LOOKBACK_BARS = 50 # bounds how
    far back a "recent" pivot can be

    swing_highs = []
    swing_lows = []
```

```

        _search_floor = max(6, len(highs_arr) -
3 - MAX_LOOKBACK_BARS)

        for i in range(len(highs_arr) - 3,
_search_floor, -1):
            if (
                highs_arr[i] > highs_arr[i - 1]
                and highs_arr[i] > highs_arr[i
- 2]
                and highs_arr[i] > highs_arr[i
- 3]
                and highs_arr[i] > highs_arr[i
- 4]
                and highs_arr[i] > highs_arr[i
+ 1]
                and highs_arr[i] > highs_arr[i
+ 2]
                and (highs_arr[i] -
max(closes_arr[i - 4:i]) >= _min_depth)
            ):

swing_highs.append(closes_arr[i])
        if len(swing_highs) >= 2:
            break

        for i in range(len(lows_arr) - 3,
_search_floor, -1):
            if (
                lows_arr[i] < lows_arr[i - 1]
                and lows_arr[i] < lows_arr[i -
2]
                and lows_arr[i] < lows_arr[i -
3]

```

```

        and lows_arr[i] < lows_arr[i -
4]
        and lows_arr[i] < lows_arr[i +
1]
        and lows_arr[i] < lows_arr[i +
2]
        and (min(closes_arr[i - 4:i]) -
lows_arr[i] >= _min_depth)
    ):

swing_lows.append(closes_arr[i])
    if len(swing_lows) >= 2:
        break

    if len(swing_highs) >= 2:
        if swing_highs[0] > swing_highs[1]:
            state.bos_detected = True
        elif swing_highs[0] <
swing_highs[1]:
            state.choch_detected = True

    if len(swing_lows) >= 2:
        if swing_lows[0] < swing_lows[1]:
            state.bos_detected = True
        elif swing_lows[0] > swing_lows[1]:
            state.choch_detected = True

except Exception as e:
    logger.error(f"[BOS/CHOCH] Swing
detection failed for {self.asset_type},
state left stale: {e}")

```

Layman: the structure-break detector now has a leash on how far back it'll look, can't silently drop one signal because another fired first, measures the close instead of the wick, and yells if it breaks instead of going quiet.

0.6 – VTM structure-aware plumbing (Tier 1.2) – D2/D3/D4 need this

Where: `veteran_trade_manager.py`, `__init__`.

```
def __init__(
    self, entry_price, side, asset,
    risk_config, high, low, close, quantity,
    volume=None, signal_details=None,
    account_risk=0.015, atr_period=14,
    trade_type="TREND",
    local_free_margin=0.0, current_ask=0.0,
    current_bid=0.0,
    min_lot_override=None,
    lot_precision_override=None,
    structure_levels_ref=None,
    entry_retest_type=None, telegram=None, #
    telegram added per 0.3
):
```

```
self.structure_levels_ref =
structure_levels_ref
self.entry_retest_type = entry_retest_type
self.telegram = telegram
```

```
self.atr_at_entry = self._calculate_atr()
self.structural_stop_level =
self.current_stop_loss
```

0.7 – Dependency registry (Tier 1.3, reshaped) – 5.1 reads this directly

```
JUDGE_SOURCE_REGISTRY = {
    "trend":      [("independent", 0.90),
("livermore_1h", 0.10)],
    "momentum":  [("independent", 0.85),
("livermore_1h", 0.15)],
    "pattern":   [("independent", 1.0)],
    "structure": [("independent", 0.85),
("livermore_1h", 0.15)],
    "volume":    [("independent", 0.85),
("livermore_1h", 0.15)],
    "reversion": [("independent", 1.0)],
}
# 0.10/0.15 are reasoned starting points,
# not measured – revisit once
# housekeeping's judge-liveness data gives
# real per-enrichment averages.
```

0.8 – The two flags carrying the specific, previously-named risk from the day-1-live decision

Why these two specifically, not just "everything's on now": both were left `false` by a session before this whole conversation, each with its own stated reason attached — `per_tick_livermore_enabled`'s comment named a 21-day observation window with structural stops enabled and no increase in premature stop-outs; that window never ran. Flipping both now means that observation happens live, on real trades, instead of before them.

```
// config.template.json (or the live
config, wherever these two actually live)
"per_tick_livermore_enabled": true,
"conviction_dying_directional_enabled":
true
```

What each one actually turns on, so it's not just "everything's live" as an undifferentiated mass:

- `per_tick_livermore_enabled` → D4 (Part 3.3) starts actually running — 1H and 4H Livermore state refreshing live during an open trade, adjusting trail and breakeven as it goes.
- `conviction_dying_directional_enabled` → MOMENTUM's 3.2c (Part 2.2) starts actually running — the NATURAL_REBOUND-specific SELL-side penalty-restore, previously dormant code with nothing exercising it.

The fastest thing to watch on these two specifically:

`check_cadence` and `check_connections` (Part 1.8)
staying clean confirms the per-tick refresh isn't itself
introducing lag or errors — that's a five-minute check,
not a 21-day one, and worth running the first day, not
just the first week.

PART 1 — SHIP-NOW (safe the moment Part 0 is applied)

1.1 — CHoCH symmetric fix

Where: `council_aggregator.py`,
`_judge_structure_bidirectional`, ~3518 and
~3606.

```
is_bullish_regime = bool((governor_data or
{}).get("is_bullish", False))
is_bearish_regime = bool((governor_data or
{}).get("is_bearish", False))
```

```
if choch_detected and is_bullish_regime:
    sell_score = min(sell_score + 0.3 *
weight, weight)
    sell_exp += " +CHoCH"
if choch_detected and is_bearish_regime:
    buy_score = min(buy_score + 0.3 *
```

```
weight, weight)
    buy_exp += " +CHoCH"
```

1.2 – USOIL Livermore `minor` recalibration

Where: `config/aggregator_presets.json`,
`LIVERMORE_PIVOTS.USOIL`.

```
"minor": 0.5
```

1.3 – CHASE-1, per-asset trend-continuation override

```
self.score_history = deque(maxlen=20)
self._chase_override_min_cycles = 10
```

```
_re_assets = _re_cfg.get("RETEST_ENGINE",
{}).get("assets", {})
_override_asset_cfg =
_re_assets.get(self.asset_type,
_re_assets.get("DEFAULT", {}))
self._chase_override_min_cycles =
int(_override_asset_cfg.get("chase_hard_override_min_cycles", 10))
```

```
"trend_maxed_buy":
_raw_buy_scores.get("trend", 0.0) >=
w_trend - 1e-9,
"trend_maxed_sell":
```

```

_raw_sell_scores.get("trend", 0.0) >=
w_trend - 1e-9,
"conviction_dying": bool(
    getattr(_composite_state,
"conviction_dying", False)
    if not isinstance(_composite_state,
dict)
    else (_composite_state or
{}).get("conviction_dying", False)
),

```

```

def _consecutive_maxed_cycles(self,
direction: str) -> int:
    trend_key = f"trend_maxed_{direction}"
    streak = 0
    for cycle in
reversed(self.score_history):
        if cycle.get(trend_key) and not
cycle.get("conviction_dying", False):
            streak += 1
        else:
            break
    return streak

```

```

_buy_overridden = False
_sell_overridden = False
if _buy_type == "CHASE_HARD":
    _buy_streak =
self._consecutive_maxed_cycles("buy")
    if _buy_streak >=
self._chase_override_min_cycles:
    _buy_overridden = True

```

```

        else:
            buy_total = 0.0
        if _sell_type == "CHASE_HARD":
            _sell_streak =
self._consecutive_maxed_cycles("sell")
            if _sell_streak >=
self._chase_override_min_cycles:
                _sell_overridden = True
            else:
                sell_total = 0.0
        if (_buy_type == "CHASE_HARD" and
            _sell_type == "CHASE_HARD"
            and not _buy_overridden and not
            _sell_overridden):
            decision_type = "HOLD
(structural_chase_hard)"

```

```

"BTC":    {
"chase_hard_override_min_cycles": 12 },
"GOLD":   {
"chase_hard_override_min_cycles": 11 },
"EURUSD": {
"chase_hard_override_min_cycles": 12 },
"EURJPY": {
"chase_hard_override_min_cycles": 10 },
"GBPAUD": {
"chase_hard_override_min_cycles": 59 },
"USTEC":  {
"chase_hard_override_min_cycles": 12 },
"USOIL":  {
"chase_hard_override_min_cycles": 10 },

```

```
"DEFAULT":{  
  "chase_hard_override_min_cycles": 10 }
```

1.4 – Tier X.2, TransitionDetector feeds the outcome pipeline

```
if hasattr(self, "outcome_pipeline"):  
    _close_now =  
float(df["close"].iloc[-1]) if df is not  
None and len(df) else None  
    if _close_now is not None:  
        self.outcome_pipeline.tag(  
            asset=self.asset_type,  
            direction="bullish" if  
"BULLISH" in _te.direction else "bearish",  
            price=_close_now,  
            timestamp=datetime.now(timezone.utc),  
            event_type="softening",  
        )
```

1.5 – Tier X.3a, composite_state capture bug

```
_aggregator =  
self.aggregators.get(asset_name)  
if isinstance(_aggregator, dict):  
    _aggregator =  
_aggregator.get("council", _aggregator)  
_comp_state_dict = {}  
if _aggregator and hasattr(_aggregator,  
'_cached_composite') and  
_aggregator._cached_composite is not None:
```

```

try:
    _comp_state_dict =
_aggregator._cached_composite.to_dict()
except Exception:
    pass

```

1.6 – Tier X.3b, six judge scores added to shadow's `strategy_votes`

```

strategy_votes={
    "mr_signal":
signal_details.get("mr_signal", 0),
    "mr_conf":
signal_details.get("mr_conf", 0.0),
    "tf_signal":
signal_details.get("tf_signal", 0),
    "tf_conf":
signal_details.get("tf_conf", 0.0),
    "ema_signal":
signal_details.get("ema_signal", 0),
    "ema_conf":
signal_details.get("ema_conf", 0.0),
    "signal_quality":
signal_details.get("signal_quality", 0.0),
    "judge_buy_scores":
signal_details.get("buy_scores", {}),
    "judge_sell_scores":
signal_details.get("sell_scores", {}),
},

```

X.3c – no code. `mr` / `tf` / `ema` confirmed real, leave as-is.

1.7 – Tier X.4, funnel_logger's ambiguous-contest bucket

```
families = {
    "stale_price": "blocked_stale_price",
    "flash_veto": "blocked_flash",
    "ny_open": "blocked_ny_open",
    "econ_calendar": "blocked_calendar",
    "no_governor": "blocked_no_governor",
    "silent zone": "blocked_silent_zone",
    "silent_zone": "blocked_silent_zone",
    "hard_veto": "blocked_hard_veto",
    "ambiguous_both_sides_cleared":
    "blocked_ambiguous_contest",
    "quality": "blocked_low_quality",
    "score": "blocked_low_score",
    "rejected": "blocked_low_score",
}
```

1.8 – Housekeeping

```
class HealthMonitor:
    def __init__(self):
        self.last_heartbeat = time.time()
        self.error_count = 0
        self.start_time = time.time()
        self.asset_cadence_baseline_min = {
            "GBPAUD": 1.1, "BTC": 5.3,
            "EURUSD": 5.3, "USTEC": 5.3, "GOLD": 6.0,
            "USOIL": 6.7,
        }
```



```

def heartbeat(self):
    self.last_heartbeat = time.time()

def record_error(self):
    self.error_count += 1
    logger.warning(f"[HEALTH] Error
recorded. Current count:
{self.error_count}")

def is_healthy(self):
    now = time.time()
    heartbeat_ok = (now -
self.last_heartbeat) < 60
    errors_ok = self.error_count < 10
    return heartbeat_ok and errors_ok

def check_judge_liveness(self,
aggregators: dict, window: int = 20) ->
dict:
    report = {}
    for asset, agg_entry in
aggregators.items():
        agg = agg_entry.get("council",
agg_entry) if isinstance(agg_entry, dict)
else agg_entry
        history = getattr(agg,
"score_history", None)
        if not history:
            report[asset] = {"status":
"no_history"}
            continue
        recent = list(history)[-
window:]
        judge_names = set()

```

```

        for cycle in recent:
            judge_names |=
set(cycle.get("buy_scores", {}).keys())
            per_judge = {}
            for j in judge_names:
                fired = sum(1 for c in
recent
                            if
c.get("buy_scores", {}).get(j, 0) > 0
                            or
c.get("sell_scores", {}).get(j, 0) > 0)
                pct = fired / len(recent)
                flag = "DEAD" if pct == 0
else "SUSPICIOUSLY_PINNED" if pct > 0.95
else "OK"
                per_judge[j] =
{"fired_pct": round(pct, 2), "flag": flag}
            report[asset] = per_judge
        return report

    def check_cadence(self, aggregators:
dict) -> dict:
        report = {}
        for asset, agg_entry in
aggregators.items():
            agg = agg_entry.get("council",
agg_entry) if isinstance(agg_entry, dict)
else agg_entry
            history = getattr(agg,
"score_history", None)
            if not history or len(history)
< 2:
                continue
            last_ts =

```

```

history[-1].get("timestamp")
        baseline_min =
self.asset_cadence_baseline_min.get(asset,
5.5)
        if last_ts:
            age_min = (time.time() -
last_ts) / 60 if isinstance(last_ts, (int,
float)) else None
            if age_min is not None:
                report[asset] = {

"minutes_since_last_cycle": round(age_min,
1),
                                "baseline_min":
baseline_min,
                                "late": age_min >
baseline_min * 3,
                                }
        return report

    def check_decision_completeness(self,
aggregators: dict, window_minutes: int =
60) -> dict:
        report = {}
        now = time.time()
        for asset, agg_entry in
aggregators.items():
            agg = agg_entry.get("council",
agg_entry) if isinstance(agg_entry, dict)
            else agg_entry
            history = getattr(agg,
"score_history", None)
            if not history:
                continue

```

```

        cutoff = now - (window_minutes
* 60)
        actual = sum(1 for c in history
if isinstance(c.get("timestamp"), (int,
float)) and c["timestamp"] >= cutoff)
        baseline_min =
self.asset_cadence_baseline_min.get(asset,
5.5)
        expected = max(1,
int(window_minutes / baseline_min))
        completeness = actual /
expected
        report[asset] = {
            "actual_cycles": actual,
            "expected_cycles": expected,
            "completeness_pct":
round(completeness * 100, 1),
            "flag": "GAPS_DETECTED" if
completeness < 0.7 else "OK",
        }
        return report

    def check_connections(self, mt5_module,
binance_handler, cvd_consumer) -> dict:
        return {
            "mt5_connected":
mt5_module.terminal_info() is not None,
            "binance_stale":
binance_handler.is_stale() if
binance_handler else None,
            "cvd_stale":
cvd_consumer.is_stale() if cvd_consumer
else None,
        }

```

```

def get_status(self):
    return {
        "healthy": self.is_healthy(),
        "uptime": time.time() - self.start_time,
        "last_heartbeat_ago":
            time.time() - self.last_heartbeat,
        "error_count": self.error_count,
    }

```

Binance liveness companion fix,

`binance_handler.py`:

```

self._last_successful_call = None

ticker =
self.client.futures_symbol_ticker(symbol=sy
mbol)
self._last_successful_call = time.time()

def is_stale(self, max_age_sec: int = 120)
-> bool:
    if self._last_successful_call is None:
        return True
    return (time.time() -
self._last_successful_call) > max_age_sec

```

Daily digest, wired to the scheduler that already exists:

```

def send_daily_health_digest(self):
    liveness =

```

```

self.health_monitor.check_judge_liveness(self.aggregators)
    cadence =
self.health_monitor.check_cadence(self.aggregators)
    completeness =
self.health_monitor.check_decision_completeness(self.aggregators)
    connections = {
        "mt5_down_since": getattr(self,
"_mt5_down_since", None),
        "binance_stale":
self.binance_handler.is_stale() if
self.binance_handler else None,
        "cvd_stale":
self.cvd_consumer.is_stale() if
self.cvd_consumer else None,
    }
    dead_judges = [f"{a}:{j}" for a, jd in
liveness.items() if isinstance(jd, dict)
                    for j, v in jd.items()
if v.get("flag") == "DEAD"]
    late_assets = [a for a, c in
cadence.items() if c.get("late")]
    gap_assets = [a for a, c in
completeness.items() if c.get("flag") ==
"GAPS_DETECTED"]
    msg = (f"Daily health digest:\nDead
judges: {dead_judges or 'none'}\n"
          f"Late cycles: {late_assets or
'none'}\nGaps detected: {gap_assets or
'none'}\n"
          f"Connections: {connections}")
    self._safe_send_telegram(msg)

```

```
schedule.every().day.at("07:00").do(self.send_daily_health_digest)
```

1.9 – RLHF-light

```
self.outcome_pipeline.record(position,  
_vtm, reason, human_label=None)
```

```
keyboard = [  
    InlineKeyboardButton("Clean, good  
result",  
callback_data=f"rlhf_label:clean_good:  
{position.db_trade_id}"),  
    InlineKeyboardButton("Lucky, weak  
process",  
callback_data=f"rlhf_label:lucky_weak:  
{position.db_trade_id}"),  
], [  
    InlineKeyboardButton("Good process, bad  
luck",  
callback_data=f"rlhf_label:good_bad_luck:  
{position.db_trade_id}"),  
    InlineKeyboardButton("Process was  
wrong", callback_data=f"rlhf_label:wrong:  
{position.db_trade_id}"),  
]  
await  
self.bot.send_message(chat_id=self.chat_id,  
text=f"{asset} closed {pnl_pct:+.1f}%". Was  
this:",
```

```
reply_markup=InlineKeyboardMarkup(keyboard)
)
```

```
elif
callback_data.startswith("rlhf_label:"):
    _, label, trade_id =
callback_data.split(":", 2)

self.outcome_pipeline.attach_human_label(trade_id, label)
    await query.edit_message_text(f"Logged:
{label.replace('_', ' ')}")
```

H3 — one daily call to the already-working
`send_notification`, no new mechanism.

1.10 — Tier 2.1, CHOCH_HOLD

```
if level is not None:
    choch_now = getattr(state,
"choch_detected", False)
    dist_atr = abs(close - level) / atr
    if choch_now and getattr(state,
"level_defended", False) and dist_atr <=
self._clean_atr_mult:
        return RetestResult(
            retest_type="CHOCH_HOLD",
            modifier=self._mod_clean - 0.1,
            entry_type=ET_RANGE_BOUNDARY,
            direction=direction,
            level=level, level_2=level_2,
```



```
level_3=level_3,  
    )
```

Verified: nothing outside `retest_engine.py` pattern-matches `"CLEAN"` as a literal string.

1.11 — Tier 6.1, corrected Triple Barrier labeler (standalone script)

```
import pandas as pd  
import numpy as np  
import glob, os  
  
TARGET_ATR, STOP_ATR, TIMEOUT_BARS = 2.0,  
1.0, 20  
MAX_LOOKBACK_BARS = 50  
  
def detect_bos_choch_at_bar(highs, lows,  
    closes, atrs, i, min_depth_mult=0.3):  
    if i < 7 or i >= len(highs) - 2:  
        return False, False  
    min_depth = min_depth_mult * atrs[i] if  
    atrs[i] > 0 else 0.0  
    floor = max(6, i - MAX_LOOKBACK_BARS)  
    swing_highs, swing_lows = [], []  
    for j in range(i - 2, floor, -1):  
        if (highs[j] > highs[j-1] and  
            highs[j] > highs[j-2] and highs[j] >  
            highs[j-3]  
            and highs[j] > highs[j-4]  
            and highs[j] > highs[j+1] and highs[j] >  
            highs[j+2])
```

```

        and (highs[j] -
max(closes[j-4:j]) >= min_depth)):
        swing_highs.append(closes[j])
        if len(swing_highs) >= 2:
            break
        for j in range(i - 2, floor, -1):
            if (lows[j] < lows[j-1] and lows[j]
< lows[j-2] and lows[j] < lows[j-3]
                and lows[j] < lows[j-4] and
lows[j] < lows[j+1] and lows[j] < lows[j+2]
                and (min(closes[j-4:j]) -
lows[j] >= min_depth)):
                swing_lows.append(closes[j])
                if len(swing_lows) >= 2:
                    break
        bos = choch = False
        if len(swing_highs) >= 2:
            bos = swing_highs[0] >
swing_highs[1] or bos
            choch = swing_highs[0] <
swing_highs[1] or choch
        if len(swing_lows) >= 2:
            bos = swing_lows[0] < swing_lows[1]
or bos
            choch = swing_lows[0] >
swing_lows[1] or choch
        return bos, choch

def label_events(df):
    df = df.reset_index(drop=True)
    atr =
df["high"].sub(df["low"]).rolling(14).mean(
).values
    highs, lows, closes =

```

```

df["high"].values, df["low"].values,
df["close"].values
    results = []
    for i in range(60, len(df) -
TIMEOUT_BARS):
        bos, choch =
detect_bos_choch_at_bar(highs, lows,
closes, atr, i)
        if not (bos or choch):
            continue
        entry = closes[i]
        target, stop = entry + TARGET_ATR *
atr[i], entry - STOP_ATR * atr[i]
        window_hi =
df["high"].iloc[i+1:i+1+TIMEOUT_BARS]
        window_lo =
df["low"].iloc[i+1:i+1+TIMEOUT_BARS]
        hit_t = window_hi.index[window_hi
>= target]
        hit_s = window_lo.index[window_lo
<= stop]
        if len(hit_t) and (not len(hit_s)
or hit_t[0] < hit_s[0]):
            outcome = "win"
        elif len(hit_s):
            outcome = "loss"
        else:
            outcome = "timeout"
        results.append({"bar": i, "event":
"BOS" if bos else "CHoCH", "outcome":
outcome})
    return pd.DataFrame(results)

if __name__ == "__main__":

```

```

    for path in
glob.glob("data/raw/*_4h.csv"):
        symbol =
os.path.basename(path).split("_")[0]
        out =
label_events(pd.read_csv(path))
        for ev in ("BOS", "CHoCH"):
            sub = out[out["event"] == ev]
            if len(sub):
                wr = (sub["outcome"] ==
"win").mean()
                print(f"{symbol} {ev}: n=
{len(sub)} win_rate={wr:.1%}")

```

Sequencing note: once shadow_data carries real judge-level context (1.6), treat it as primary for grading real blocked signals.

PART 2 — TIER 3: THE JUDGES, DECOUPLED (live from day one, watched continuously per Part 5)

2.1 — TREND (function + call site)

```

def _judge_trend_bidirectional(self, df,
is_bull, weight, ema_signal=0,
ema_conf=0.0, consensus_regime="NEUTRAL",
governor_data=None):
    try:

```

```

        buy_score = weight * ema_conf if
ema_signal == 1 else 0.0
        sell_score = weight * ema_conf if
ema_signal == -1 else 0.0
        buy_exp = f"TREND BUY: {'OK' if
buy_score else 'NO'} EMA aligned
({buy_score:.2f})"
        sell_exp = f"TREND SELL: {'OK' if
sell_score else 'NO'} EMA aligned
({sell_score:.2f})"

        lv_1h = (governor_data or
{}).get("composite_state")
        lv_state = getattr(lv_1h,
"livermore_state_1h", None) if lv_1h and
not isinstance(lv_1h, dict) else
(lv_1h.get("livermore_state_1h") if
isinstance(lv_1h, dict) else None)
        _bull_states = ("MAIN_UP",
"NATURAL_RETRACEMENT",
"SECONDARY_RETRACEMENT")
        _bear_states = ("MAIN_DOWN",
"NATURAL_REBOUND", "SECONDARY_REBOUND")
        if buy_score > 0 and lv_state in
_bull_states:
            buy_score = min(buy_score *
1.15, weight)
            if sell_score > 0 and lv_state in
_bear_states:
                sell_score = min(sell_score *
1.15, weight)
            return buy_score, sell_score,
{"buy": buy_exp, "sell": sell_exp}
        except Exception as e:

```

```

        logger.error(f"[TREND] Error: {e}")
        return 0.0, 0.0, {"buy": "TREND:
Error", "sell": "TREND: Error"}

```

```

buy_scores["trend"], sell_scores["trend"],
trend_exp = (
    self._judge_trend_bidirectional(
        df, is_bull, w_trend,
        ema_signal=ema_signal, ema_conf=ema_conf,
        consensus_regime=consensus_regime,
        governor_data=governor_data
    )
)

```

2.2 – MOMENTUM

```

base = weight * min(_adx_for_gate / 32.0,
1.0)

_slope = compute_adx_slope(_adx_col)
_regime = _slope["regime"]
_SLOPE_DELTA = {"RISING_FAST":
+0.20*weight, "RISING": +0.10*weight,
"FLAT": 0.0,
                "FALLING": -0.15*weight,
"FALLING_FAST": -0.25*weight}
_delta = _SLOPE_DELTA.get(_regime, 0.0)
_delta_source = "dedicated"

_lsm_1h = _cs("livermore_state_1h", None)
if cs else None
if _lsm_1h in ("SECONDARY_RETRACEMENT",

```

```

"SECONDARY_REBOUND"):
    _delta = -_delta
    _delta_source = "Livermore-
flipped(SECONDARY)"
elif _lsm_1h in ("NATURAL_RETRACEMENT",
"NATURAL_REBOUND") and _delta > 0:
    _delta = 0.0
    _delta_source = "Livermore-
neutralized(NATURAL)"

penalty = 0.20 * weight
buy_score = max(0.0, buy_score - penalty)
sell_score = max(0.0, sell_score - penalty)
buy_exp += " -conviction_dying(dedicated)"
sell_exp += " -conviction_dying(dedicated)"

_cd_dir = self.config.get("phase_config",
{}).get("conviction_dying_directional_enabl
ed", False)
if _cd_dir and _lsm_1h ==
"NATURAL_REBOUND":
    sell_score = min(sell_score + penalty,
weight)
    sell_exp += " +Livermore-
override(NATURAL_REBOUND, SELL restored)"

```

2.3 – PATTERN

```

_close = df["close"].iloc[-1]
_supp = getattr(state,
"nearby_support_level", None)
_res = getattr(state,

```

```

"nearby_resistance_level", None)
_dist_supp = abs(_close - _supp) if _supp
is not None else float("inf")
_dist_res = abs(_close - _res) if _res is
not None else float("inf")
_basing_bullish = _dist_supp < _dist_res
_basing_bearish = _dist_res < _dist_supp

if _basing_bullish and (is_tight_range or
vol_ratio > 1.3):
    _conf = 0.3 + 0.2 * min(1.0, atr /
max(_dist_supp, 1e-9))
    state.institutional_pattern,
state.institutional_pattern_confidence =
"ACCUMULATION", _conf
elif _basing_bearish and (is_tight_range or
vol_ratio > 1.3):
    _conf = 0.3 + 0.2 * min(1.0, atr /
max(_dist_res, 1e-9))
    state.institutional_pattern,
state.institutional_pattern_confidence =
"DISTRIBUTION", _conf
elif is_tight_range:
    state.institutional_pattern,
state.institutional_pattern_confidence =
"COMPRESSION", 0.3
else:
    state.institutional_pattern,
state.institutional_pattern_confidence =
None, 0.0

```

2.4 – VOLUME


```

def _apply_volume_divergence_bonus(self,
buy_score, sell_score, buy_exp, sell_exp,
cs, weight):
    if cs is None:
        return buy_score, sell_score,
buy_exp, sell_exp
    _get = (lambda k, d=None: cs.get(k, d))
    if isinstance(cs, dict) else (lambda k,
d=None: getattr(cs, k, d))
    _div_bonus = 0.10
    _lsm_1h = _get("livermore_state_1h")
    _lsm_topup = 0.0
    if _lsm_1h in ("NATURAL_RETRACEMENT",
"NATURAL_REBOUND"):
        _lsm_topup = 0.10
    elif _lsm_1h in
("SECONDARY_RETRACEMENT",
"SECONDARY_REBOUND"):
        _lsm_topup = 0.15
    _total = _div_bonus + _lsm_topup
    if bool(_get("bullish_divergence",
False)):
        buy_score = min(buy_score + _total
* weight, weight)
        buy_exp += f"
+bull_divergence(dedicated=
{_div_bonus:.2f}, Livermore+
{_lsm_topup:.2f})"
    if bool(_get("bearish_divergence",
False)):
        sell_score = min(sell_score +
_total * weight, weight)
        sell_exp += f"

```

```
+bear_divergence(dedicated=
{_div_bonus:.2f}, Livermore+
{_lsm_topup:.2f})"
    return buy_score, sell_score, buy_exp,
sell_exp
```

PART 3 – TIER 4: TRUST LAYER

(depends on Part 2 being live)

3.1 – RetestEngine's BREAKOUT tier

```
age_1h = int(getattr(state,
"livermore_state_age_1h", 999))
bo_level = self._get_breakout_level(state,
direction)
if bo_level is not None:
    dist_atr = abs(close - bo_level) / atr
    prox_mult =
float(asset_cfg.get("breakout_proximity_atr
_mult", 2.0 if self._is_btc(symbol) else
1.25))
    if dist_atr <= prox_mult:
        mod =
self._breakout_modifier(symbol, state,
direction)
        _fresh = age_1h <=
self._breakout_age_max
        if not _fresh:
            mod += 0.15
    return
```

```

RetestResult(retest_type=RT_BREAKOUT,
modifier=mod,
entry_type=ET_TREND_FOLLOWING,

direction=direction, level=bo_level,
level_2=level_2, level_3=level_3)

```

Flag for Stephen: makes CONTINUATION's stale-case largely unreachable — decide whether to remove it or keep for a future purpose.

3.2 — D2/D3 (in `portfolio_manager.py`)

```

if getattr(_vtm, "structural_stop_level",
None) is not None and getattr(_vtm,
"structure_levels_ref", None) is not None:
    _atr_entry = getattr(_vtm,
"atr_at_entry", None) or 1e-9
    _level_held = reason !=
ExitReason.STOP_LOSS.value
    for lvl in _vtm.structure_levels_ref:
        if abs(lvl["price"] -
_vtm.structural_stop_level) < 0.1 *
_atr_entry:
            if _level_held:
                lvl["tests"] =
lvl.get("tests", 0) + 1
            else:
                lvl["failed_tests"] =
lvl.get("failed_tests", 0) + 1
            if getattr(_vtm,
"entry_retest_type", None) == "CLEAN" and

```

```

reason in (

ExitReason.TAKE_PROFIT_1.value,
ExitReason.TAKE_PROFIT_2.value,
ExitReason.TAKE_PROFIT_3.value,
):

lvl["clean_retest_success_count"] =
lvl.get("clean_retest_success_count", 0) +
1

```

3.3 – D4

```

self.livermore_state_1h =
_cs.get("livermore_state_1h")
self.livermore_state_age_1h =
int(_cs.get("livermore_state_age_1h", 0))

```

```

_new_lv1h = getattr(composite_state,
"livermore_state_1h", None)
if _new_lv4h and _new_lv4h !=
self.livermore_state_4h:
    if _new_lv1h:
        self.livermore_state_1h = _new_lv1h

```

```

elif prev_state in _MAIN and new_state in
_MAIN and prev_state != new_state:
    thesis_confirmed = (is_long and
new_state == "MAIN_UP") or (not is_long and
new_state == "MAIN_DOWN")
    lv_1h_confirmed = (is_long and

```

```

self.livermore_state_1h == "MAIN_UP") or
(not is_long and self.livermore_state_1h ==
"MAIN_DOWN")
    if thesis_confirmed:
        self.runner_trail_atr_multiplier =
_config_trail * _main_trail
        self._lv_breakeven_locked = False
    else:
        self.runner_trail_atr_multiplier =
_config_trail * _sec_trail * 0.3
        if not getattr(self,
"_lv_breakeven_locked", False):
            self.current_stop_loss =
self.entry_price
            self._lv_breakeven_locked =
True

```

3.4 – Continuous whole-panel monitoring

```

def update_with_current_price(self,
current_price, df_4h=None,
composite_state=None, judge_scores=None):
    self._live_cs = composite_state
    self._live_judge_scores = judge_scores

```

```

_fresh_signal, _fresh_details =
self.aggregators[asset_name]
["council"].get_aggregated_signal(df, ...)
position.trade_manager.update_with_current_
price(
    current_price, df_4h=df_4h,
    composite_state=composite_state,

```

```

judge_scores=_fresh_details.get("buy_scores", {}) | _fresh_details.get("sell_scores", {}),
)

```

3.5 – Human-alert layer

```

def _check_alert_conditions(self):
    if self._live_cs is None:
        return None
    _cs = self._live_cs
    _against = "sell" if self.side == "long" else "buy"
    signals_fired = []
    if getattr(_cs, "choch_detected", False):
        signals_fired.append("ChoCH against position")
    if getattr(_cs, "bos_detected", False) and self._live_judge_scores and self._live_judge_scores.get(_against, {}).get("structure", 0) > 0:
        signals_fired.append("structure break against position")
    if getattr(_cs, "conviction_dying", False):
        signals_fired.append("conviction fading")
    if getattr(_cs, "bearish_divergence" if self.side == "long" else "bullish_divergence", False):

```

```
        signals_fired.append("momentum
divergence against position")
        if len(signals_fired) >= 3:
            return f"{self.asset} {self.side}:
{len(signals_fired)} signals against - " +
", ".join(signals_fired)
        return None

def _safe_send_alert(self, message):
    if not self.telegram or not
self.telegram.is_running:
        return
    try:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)

        loop.run_until_complete(self.telegram.send_
notification(message))
    except Exception as e:
        logger.error(f"[VTM ALERT] Failed
to send: {e}")
```

PART 4 – TIER 5: ARBITRATION

**(depends on Part 0's reshaped
registry)**

4.1 – Correlation discount

```

def _effective_vote_count(self,
contributing_judges, registry):
    seen_shared_fields = {}
    effective = 0.0
    for judge in contributing_judges:
        entries = registry.get(judge,
[("independent", 1.0)])
        judge_total = 0.0
        for source, share in entries:
            if source == "independent":
                judge_total += share
            else:
                prior_use =
seen_shared_fields.get(source, 0.0)
                judge_total += share *
max(0.0, 1.0 - prior_use)
                seen_shared_fields[source]
= min(1.0, prior_use + share)
            effective += judge_total
    return effective

```

```

_buy_contributing = [j for j in
JUDGE_SOURCE_REGISTRY if buy_scores.get(j,
0) > 0]
_sell_contributing = [j for j in
JUDGE_SOURCE_REGISTRY if sell_scores.get(j,
0) > 0]
_buy_eff_n =
self._effective_vote_count(_buy_contributin
g, JUDGE_SOURCE_REGISTRY)
_sell_eff_n =
self._effective_vote_count(_sell_contributi

```



```

ng, JUDGE_SOURCE_REGISTRY)
_buy_scale = (_buy_eff_n /
len(_buy_contributing)) if
_buy_contributing else 1.0
_sell_scale = (_sell_eff_n /
len(_sell_contributing)) if
_sell_contributing else 1.0
_buy_score_pct = (buy_total * _buy_scale
/ _achievable_max) if _achievable_max > 0
else 0.0
_sell_score_pct = (sell_total * _sell_scale
/ _achievable_max) if _achievable_max > 0
else 0.0

```

4.2 – "Wait for clarity"

```

elif _buy_clears and _sell_clears:
    _price_action_leans_buy =
buy_scores.get("structure", 0) >
sell_scores.get("structure", 0)
    _price_action_leans_sell =
sell_scores.get("structure", 0) >
buy_scores.get("structure", 0)
    if _price_action_leans_buy and not
_price_action_leans_sell:
        signal, total_score,
required_score, chosen_scores = (1,
buy_total, _buy_threshold, buy_scores)
    elif _price_action_leans_sell and not
_price_action_leans_buy:
        signal, total_score,
required_score, chosen_scores = (-1,

```

```

sell_total, _sell_threshold, sell_scores)
    else:
        signal, total_score,
        required_score, chosen_scores = (0, 0.0,
        max(_buy_threshold, _sell_threshold), {})
        decision_type = "HOLD
        (ambiguous_both_sides_cleared)"

```

5.3 — no new code, cross-references 1.4.

PART 5 — DAY-1 LIVE MONITORING

**(same checks as before, now
watching a live system instead of
gating one)**

**5.1 — Mechanics, flipped: new judges drive
real decisions by default now, old ones are the
reference**

```

_tier3_shadow_mode =
self.config.get("phase_config",
{}).get("tier3_shadow_enabled", False) #
now defaults False – new judges live from
day 1
_old_buy, _old_sell, _ =
self._judge_trend_bidirectional_legacy(df,
is_bull, w_trend, consensus_regime,
governor_data=governor_data)

```

```

_new_buy, _new_sell, _ =
self._judge_trend_bidirectional(df,
is_bull, w_trend, ema_signal=ema_signal,
ema_conf=ema_conf,
consensus_regime=consensus_regime,
governor_data=governor_data)
buy_scores["trend"], sell_scores["trend"] =
(_old_buy, _old_sell) if _tier3_shadow_mode
else (_new_buy, _new_sell)

```

Same pattern for MOMENTUM and PATTERN. **The _legacy functions stay in the file, computed every cycle, driving nothing** — purely so there's a real, live reference to compare against while watching the new judges trade. Cheap to keep, valuable the moment something looks off.

This one flag is the emergency brake, worth repeating from the top of this file: flip

tier3_shadow_enabled to True and every Tier 3 judge instantly reverts to the old, already-proven behavior — no redeploy. That's the fast first move if 5.2 or 5.4 below show something wrong; the actual bug still gets fixed properly afterward, but real decisions stop being affected within one cycle.

5.2 — Logging, corrected for which side is now live vs. reference

```

self.score_history.append({
    "timestamp": timestamp,

```

```

    "buy_scores": _raw_buy_scores,
    "sell_scores": _raw_sell_scores,    # now
the NEW judges by default – this is what's
live
    "legacy_reference_buy_scores": {
        "trend": _old_buy_trend,
    "momentum": _old_buy_momentum, "pattern":
_old_buy_pattern,
    },
    "legacy_reference_sell_scores": {
        "trend": _old_sell_trend,
    "momentum": _old_sell_momentum, "pattern":
_old_sell_pattern,
    },
    "buy_total":
sum(_raw_buy_scores.values()),
    "sell_total":
sum(_raw_sell_scores.values()),
    })

```

Part 1.8's `check_judge_liveness` **already reads** `buy_scores` / `sell_scores` **directly** — with this flip, it's automatically watching the new judges now, since that's what populates those fields by default. No separate "shadow liveness" check needed anymore for that purpose; the existing housekeeping check does the job, pointed at whatever's actually live. Given there's no pre-launch window this time, **this is worth watching daily at minimum, not just glancing at the digest** — a DEAD or SUSPICIOUSLY_PINNED flag on TREND, MOMENTUM, or PATTERN in the first days is

the clearest, fastest signal something needs the emergency brake.

5.3 – Divergence tagging, unchanged mechanically, now the primary bug-detection tool instead of a pre-launch measurement

```
if _old_final_signal != _new_final_signal:
    details["tier3_divergence"] = {
        "signal_to_shadow":
        _new_final_signal, # always the new signal
        now – it's what's actually live
        "gate_label":
        f"tier3_divergence_live_{self.asset_type}",
    }
```

```
if details.get("tier3_divergence"):
    _div = details["tier3_divergence"]
    self._shadow_open_blocked(asset_name,
    _div["signal_to_shadow"], details, df,
    current_price, _div["gate_label"],
    asset_cfg)
```

Every cycle where the new judges would have called it differently than the old ones now gets tagged and tracked with a real synthetic outcome – same mechanism as before, just running continuously against live trading instead of as a pre-launch measurement.

5.4 — Grading, reframed from "prove it before going live" to "keep proving it while it's live"

```
def
grade_tier3_live_performance(shadow_records
, min_n=15):
    """No longer a go/no-go gate — this
    runs continuously now. A result
    showing the new side losing to the
    reference isn't a reason to wait
    and see; it's the trigger to flip
    tier3_shadow_enabled back to True
    while the actual issue gets
    diagnosed."""
    live_side = [r for r in shadow_records
if r.get("gate_blocked_by",
"").startswith("tier3_divergence_live")]
    if len(live_side) < min_n:
        return {"n": len(live_side),
"status":
"insufficient_sample_still_forming"}
    wins = sum(1 for r in live_side if
r.get("net_pnl_pct", 0) > 0)
    return {"n": len(live_side),
"win_rate": wins / len(live_side)}
```

5.5 — What to actually watch, given there's no pre-launch window this time

No go-live gate anymore — there's nothing left to gate.
What replaces it: **check 5.2's judge liveness and 5.4's live win rate daily for at least the first two weeks, not**

just via the once-a-day digest from Part 1.8. Before ~15 real divergence events accumulate per asset, 5.4's number is noise, not signal — don't overreact to one or two early bad outcomes before there's enough of them to mean anything. After that threshold, a live win rate clearly below what the old judges were managing, on the same divergence cases, is the real trigger for the emergency brake — not a hunch, an actual comparable number.

PART 6 — HELD BACK ON PURPOSE (not part of this deployment)

The zone ladder. Fully designed and verified — horizontal zones, ascending/descending channel lines with real slope math, MA-flip lines, legacy-governs, mark-then-confirm, close-based breaks. Not held back for risk reasons like everything else in this file — it simply isn't wired into the live pipeline at all yet. Worth revisiting once Part 5's live monitoring has run long enough to be worth adding a second new thing on top of.

FINAL VALIDATION NOTE

Two passes were run across this entire file: one checking every fix's own logic, one checking every claimed connection between pieces built in separate sessions — which is where all of Part 0 was actually found. Where something couldn't be fully verified (the CONTINUATION-tier overlap in 3.1, the exact numbers in the dependency registry), that's stated plainly rather than hidden.

That verification is what's been done. It is not the same thing as a live track record, and this file doesn't pretend otherwise. Everything here goes live on day one by deliberate choice — the bet being made is that the checking already done is worth more than a pre-launch waiting period, not that no bugs remain. Part 5 exists specifically because more probably do. The emergency brake (`tier3_shadow_enabled`) is the safety net for that bet, not a formality — know where it is before day one starts, not after something's already gone wrong.